

MCMC

Metropolis-Hastings

Grupo no sé cuánto



23/6/26

Presentan

Nicola Petetta

Jeremías Olivieri

Full Name 2



Contenidos

1 OBJETIVO



2 CADENAS DE MARKOV



3 MCMC



4 AlgoritmoS de MH



5 Ejemplo del challenger



6 Conclusion



Objetivo

> Complejidad del Objetivo: En estadística moderna, a menudo queremos muestrear de una distribución objetivo f que es altamente compleja o de gran dimensión

> El Obstáculo de la Normalización: Muchas veces solo conocemos la forma funcional de f "hasta una constante multiplicativa", lo que impide el muestreo directo

> El Teorema Ergódico es la pieza clave que valida el uso de MCMC, ya que actúa como la versión para cadenas de Markov de la Ley Fuerte de los Grandes Números

> Cambio de Paradigma: Pasamos de generar variables independientes (iid) a generar variables correlacionadas a través de una cadena de Markov

¿Qué es una Cadena de Markov?

Es una secuencia de variables aleatorias dependientes donde el futuro $x^{(t+1)}$ depende exclusivamente del estado actual $x^{(t)}$, ignorando toda la historia previa

¿Qué es una Cadena de Markov?

Es una secuencia de variables aleatorias dependientes donde el futuro $x^{(t+1)}$ depende exclusivamente del estado actual $x^{(t)}$, ignorando toda la historia previa

Distribución Estacionaria



Es el estado de equilibrio de la cadena. Si la cadena alcanza esta distribución f , se mantendrá en ella indefinidamente.

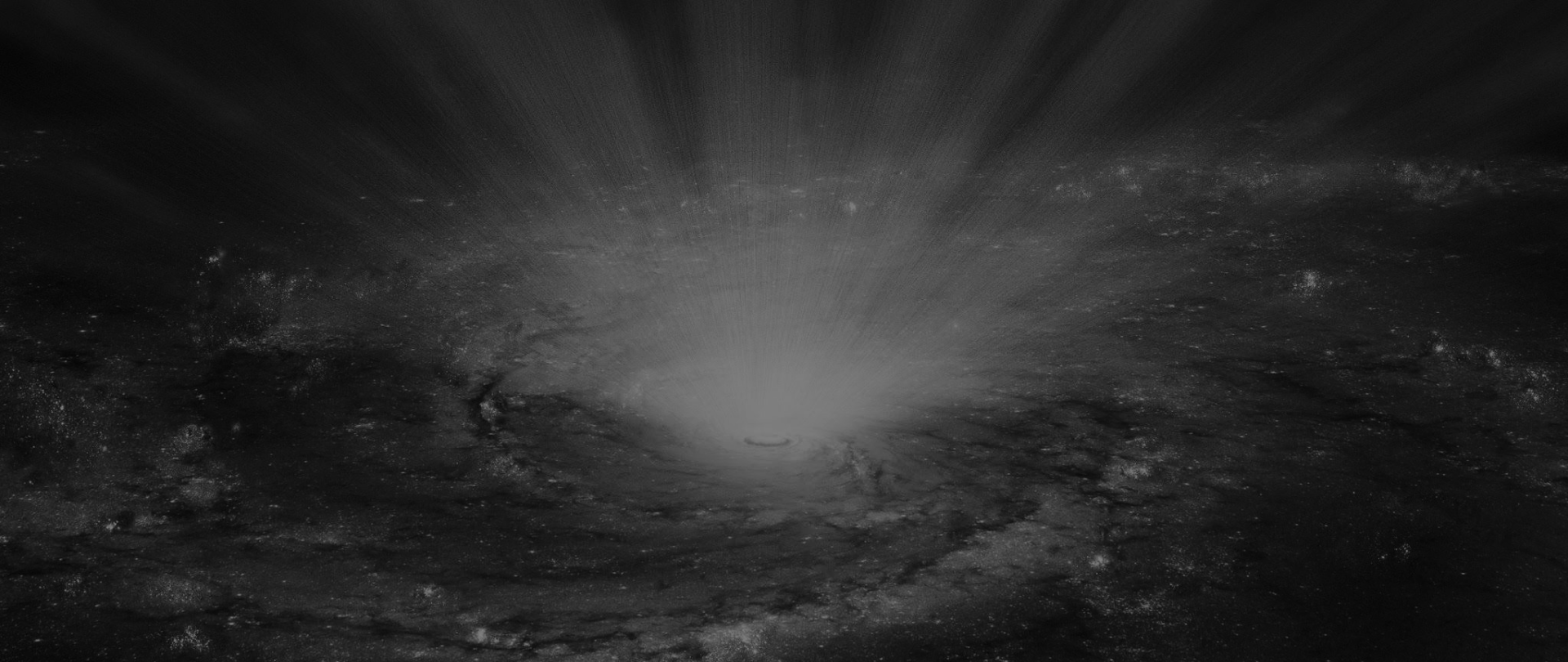
Requisitos de Estabilidad



Para que sea útil, la cadena debe ser irreducible (capaz de visitar todo el espacio) y ergódica (capaz de olvidar su estado inicial para converger al equilibrio)

¿Qué es MCMC?

- Es una clase de algoritmos que simulan variables aleatorias de una distribución objetivo compleja f mediante la construcción de una cadena de Markov, llegando a que la distribución de equilibrio sea f .
- La Inversión del Problema: En lugar de analizar una cadena existente, MCMC consiste en construir deliberadamente una cadena cuya distribución estacionaria sea exactamente la distribución objetivo f que queremos muestrear.
- ¿Por qué es potente? Funciona de forma universal para casi cualquier densidad f , incluso si no conocemos su constante de normalización, algo que detiene a los métodos clásicos
- ¿Qué es la constante de normalización? Es el factor matemático necesario para que la integral de una función sea exactamente 1, convirtiéndola en una densidad de probabilidad propia. En distribuciones complejas, calcular esta constante lleva un gran costo computacional, la ventaja de Metropolis-Hastings es que no depende de las constantes de normalización



Metropolis Hastings Generico

•

▪

Metropolis Hastings Genérico

$$Y_t \sim q(y|x^{(t)})$$

Funcionamiento: En cada paso, se genera un candidato Y_t desde q . Este se acepta como el nuevo estado de la cadena con una probabilidad específica p

$$X^{t+1} = \begin{cases} Y_t & \text{with probability } \rho(x^t, Y_t) \\ x^t & \text{with probability } 1 - \rho(x^t, Y_t) \end{cases}$$

$$\rho(x, y) = \min \left\{ \frac{f(x)}{f(y)} \frac{q(y | x)}{q(x | y)}, 1 \right\}$$

```
# Lógica general (iteración t)
y = geneq(x[t]) # Generar candidato
de q(y|x)
```

```
# Calcular ratio de aceptación
ratio = (f(y) * q(x[t], y)) / (f(x[t]) * q(y,
x[t]))
```

```
if (runif(1) < ratio) {
  x[t+1] = y # Aceptar movimiento
} else {
  x[t+1] = x[t] # Rechazar (la cadena
se mantiene constante)
}
```

CÓDIGO EN R PARA MH GENERICO y graficos??

A generic R implementation is straightforward, assuming a generator for $q(y|x)$ is available as `geneq(x)`. If $x[t]$ denotes the value of $X(t)$,

```
> y=geneq(x[t])
> if (runif(1)<f(y)*q(y,x[t])/(f(x[t])*q(x[t],y))) {
+ x[t+1]=y
+ }else{
+ x[t+1]=x[t]
+ }
```

Importancia de la Duplicación: El texto subraya que estas repeticiones son esenciales. Si borras los valores duplicados, la muestra deja de representar correctamente a la distribución objetivo

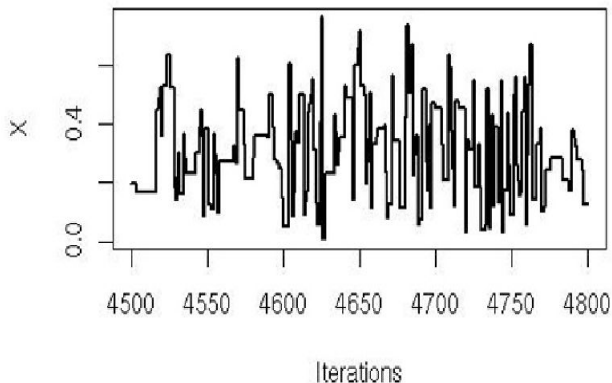


Fig. 6.1. Sequence $X^{(t)}$ for $t = 4500, \dots, 4800$, when simulated from the Metropolis-Hastings algorithm with uniform proposal and $\text{Be}(2.7, 6.3)$ target.



M-H Independiente

MH INDEPENDIENTE

1



Es un caso especial donde la densidad de propuesta $g(y)$ no depende del estado actual de la cadena $(X(t))$.

Relación con Accept-Reject: Se puede ver como una generalización del método Accept-Reject. A diferencia de este, no requiere conocer una constante M que acote la razón de densidades.

Probabilidad de Aceptación (p): $p(x,y)=\min\{f(x)f(y)g(y)g(x),1\}$

Requisito de Desempeño: Para que sea eficiente, la propuesta g debe tener colas más gruesas que la distribución objetivo f para evitar quedar "atrapada" en valores extremos

2

Algorithm 5 Independent Metropolis–Hastings

Given $x^{(t)}$

1. Generate $Y_t \sim g(y)$.

2. Take

$$X^{(t+1)} = \begin{cases} Y_t & \text{with probability } \min \left\{ \frac{f(Y_t) g(x^{(t)})}{f(x^{(t)}) g(Y_t)}, 1 \right\} \\ x^{(t)} & \text{otherwise.} \end{cases}$$



```
normal
+
rho=dt(Y,1)*dnorm(X[t-1])/(dt(
X[t-1],1)*dnorm(Y))
+ X[t]=X[t-1] +
(Y-X[t-1])*(runif(1)<rho)
+ }
```

MH INDEPENDIENTE

Es un caso especial donde la densidad de propuesta $g(y)$ no depende del estado actual de la cadena $X(t)$.

Relación con Accept-Reject: Se puede ver como una generalización del método Accept-Reject. A diferencia de este, no requiere conocer una constante M que acote la razón de densidades.

Probabilidad de Aceptación $\rho(x, y) = \min \left\{ \frac{f(x)}{f(y)} \frac{g(y)}{g(x)}, 1 \right\}$

Requisito de Desempeño: Para que sea eficiente, la propuesta g debe tener colas más gruesas que la distribución objetivo f para evitar quedar "atrapada" en valores extremos

1 Generamos $Y_t \sim g(y)$

2 Calculamos X^{t+1}

$$X^{(t+1)} = \begin{cases} Y_t & \text{con probabilidad } \min \left\{ \frac{f(Y_t)g(x^{(t)})}{f(x^{(t)})g(Y_t)}, 1 \right\} \\ x^{(t)} & \text{de lo contrario} \end{cases}$$

CÓDIGO EN R MH INDEPENDIENTE

Ejemplo 6.2 (generar una Cauchy con propuesta Normal):

```
# Paso de iteración (t)
```

```
Y = rnorm(1) # Candidato independiente de X[t-1]
```

```
# Ratio de aceptación (f es Cauchy, g es Normal)
```

```
rho = (dt(Y, 1) * dnorm(X[t-1])) / (dt(X[t-1], 1) * dnorm(Y))
```

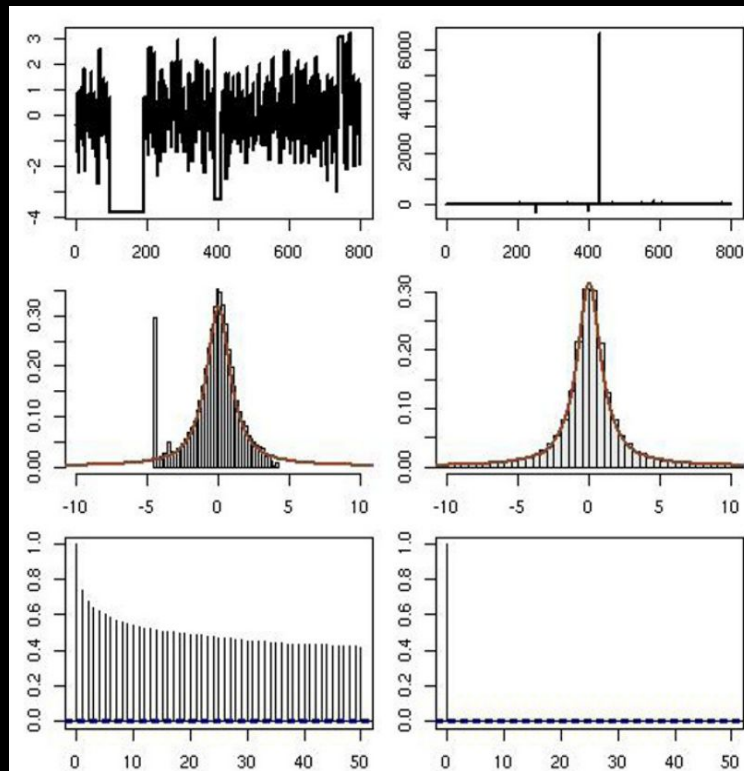
```
if (runif(1) < rho) {
```

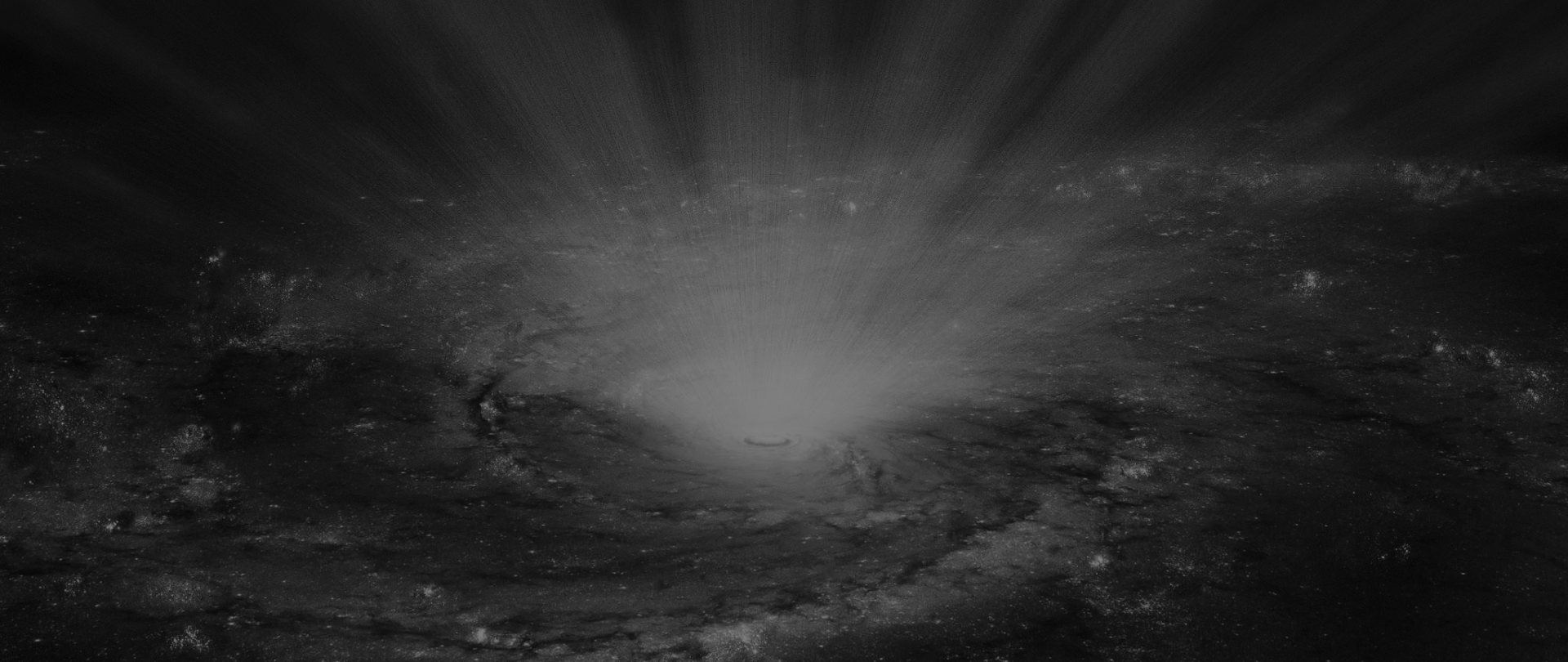
```
  X[t] = Y # Aceptar candidato
```

```
} else {
```

```
  X[t] = X[t-1] # Rechazar (duplicar valor actual)
```

```
}
```





M-H Aleatorio (Random Walk)

MH RANDOM WALK

Algorithm 6 Random walk Metropolis–Hastings

Given $x^{(t)}$,

1. Generate $Y_t \sim g(y - x^{(t)})$.

$$X^{(t+1)} = \begin{cases} Y_t & \text{con probabilidad } \min \left\{ \frac{f(Y_t)}{f(x^{(t)})}, 1 \right\} \\ x^{(t)} & \text{de lo contrario} \end{cases}$$

2. Take

$$X^{(t+1)} = \begin{cases} Y_t & \text{with probability } \min \left\{ 1, f(Y_t)/f(x^{(t)}) \right\}, \\ x^{(t)} & \text{otherwise.} \end{cases}$$

$$p(x,y)=\min\{1,f(x)f(y)\}$$

Mecanismo de generación de perturbaciones simétricas (como un $\epsilon \sim g$).
Simetría de la perturbación: $q(y|x)=q(x|y)$.
Probabilidades de aceptación: Debido a la simetría, la fórmula se simplifica y no depende de g :

mezcla de

con escala

, sd = delta)

(f es la

$X[t-1])$ {

ar movimiento

zar (el valor

la perturbación

ena se estanca

por rechazos, si es muy pequeña, se mueve demasiado lento

MH RANDOM WALK

Ejemplo 6.5 para una mezcla de normales:

1 Mecanismo: El candidato Y_t se genera sumando una perturbación aleatoria ϵ_t al estado actual: $Y_t = X(t) + \epsilon_t$, donde $\epsilon_t \sim g$.

Simetría: Si la densidad de la perturbación g es simétrica (como una Normal o Uniforme centrada en cero), entonces $q(y|x) = q(x|y)$.

Probabilidad de Aceptación (p): Debido a la simetría, la fórmula se simplifica y no depende de g :

2

$$p(x,y) = \min\{1, f(x)f(y)\}$$

Paso de iteración (t) con escala 'delta'

$prop = X[t-1] + rnorm(1, sd = delta)$

Ratio de aceptación (f es la densidad objetivo)

if (runif(1) < f(prop) / f(X[t-1])) {

$X[t] = prop$ # Aceptar movimiento

} else {

$X[t] = X[t-1]$ # Rechazar (el valor actual se duplica)

}

Nota: Si la escala de la perturbación es muy grande, la cadena se estanca por rechazos; si es muy pequeña, se mueve demasiado lento

MH RANDOM WALK

El candidato Y_t se genera sumando una perturbación aleatoria ϵ_t al estado actual

$$Y_t = X(t) + \epsilon_t, \text{ donde } \epsilon_t \sim g$$

Tiene simetría tal que la densidad de la perturbación g es simétrica (como una Normal o Uniforme centrada en cero), entonces

$$q(y | x) = q(x | y)$$

Probabilidad de Aceptación (ρ): Debido a la simetría, la fórmula se simplifica y no depende de g .

$$\rho(x, y) = \min \left\{ 1, \frac{f(x)}{f(y)} \right\}$$

1 Generamos $Y_t \sim g(y - x^t)$

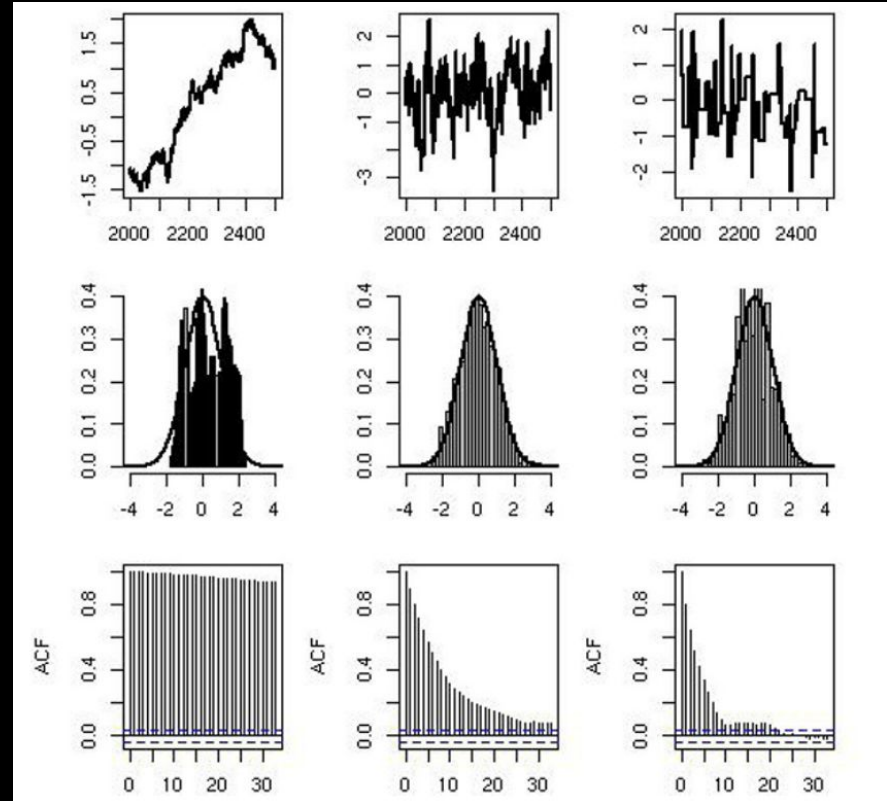
2 Calculamos X^{t+1}

$$X^{(t+1)} = \begin{cases} Y_t & \text{con probabilidad } \min \left\{ \frac{f(Y_t)}{f(x^{(t)})}, 1 \right\} \\ x^{(t)} & \text{de lo contrario} \end{cases}$$

CODIGO R RANDOM WALK

Figura 6.7 (Calibración): Muestra cómo una escala pequeña ($\delta=0.1$) genera alta autocorrelación, mientras que una escala óptima ($\delta=1$) permite que la cadena explore mejor el soporte de la normal.

Figura 6.8 (Exploración de Modos): En distribuciones con varios picos (modos), la escala debe ser lo suficientemente grande para permitir que la cadena "salte" de un modo a otro, aunque esto baje la tasa de aceptación





Algoritmo de Langevin

MH LANGEVIN

Ejemplo 6.6 (modelo Probit):

```
# Cálculo de la propuesta con deriva  
(drift)  
prop = curmean + scale * rnorm(2)  
# El promedio de la propuesta incluye el  
gradiente  
propmean = prop + 0.5 * scale^2 *  
grad(prop)  
  
# Ratio de aceptación (incluye términos  
del gradiente)  
if (log(runif(1)) > f(prop) - f(actual) +  
    sum(dnorm(actual, mean=propmean,  
sd=scale, log=T)) -  
    sum(dnorm(prop, mean=actualmean,  
sd=scale, log=T))) {  
    prop = actual # Rechazar  
}
```



MH LANGEVIN

Es una mejora de la caminata aleatoria que utiliza el **gradiente** de la densidad objetivo $\nabla \log f$ para "empujar" las propuestas hacia regiones de mayor probabilidad.

Se propone

$$Y_t = X^{(t)} + 2\sigma^2 \nabla \log f^{(t)} + \nabla \epsilon_t, \text{ donde } \epsilon_t \sim g.$$

De esta manera, el algoritmo explora el espacio de forma más eficiente y dirigida que una caminata aleatoria simple, pero a diferencia de los métodos de gradiente estocástico, el escalamiento σ es fijo y se incluye un paso de aceptación de Metropolis-Hastings para garantizar la **estacionariedad** de f .

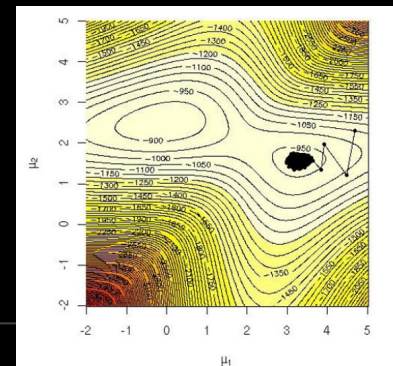
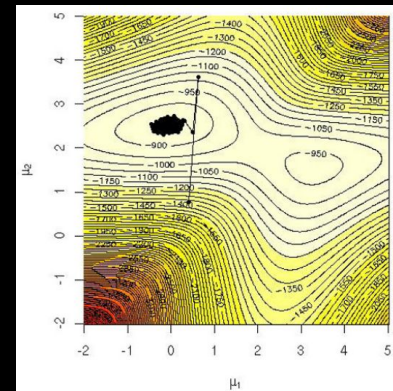
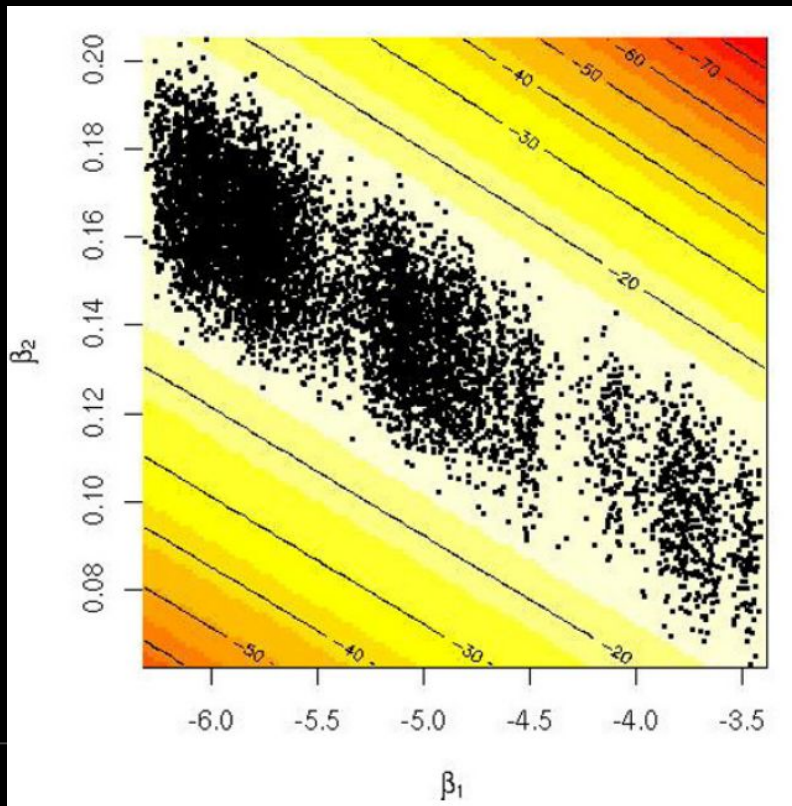
$$\rho(x, y) = \min \left\{ \frac{f(y)}{f(x)} \frac{g[(x - y)/\sigma - \sigma \nabla \log f(y)/2]}{g[(y - x)/\sigma - \sigma \nabla \log f(x)/2]}, 1 \right\}$$

R LANGEVIN

Figura 6.9 (Éxito): Muestra cómo la cadena Langevin explora correctamente la distribución posterior de un modelo Probit. Aunque se mueve lentamente, sigue la forma de la densidad de manera dirigida.

Figura 6.10 (Riesgo): En distribuciones multimodales (mezclas), el algoritmo es **muy sensible**. Si el gradiente es muy fuerte, la cadena puede quedar atrapada en un modo local ("polarización"), dificultando el salto hacia otras regiones de interés.

Moraleja: El ajuste de la escala (σ) es crítico: si es muy grande, el gradiente puede volverse inestable (NaN); si es muy pequeño, la cadena mezcla demasiado lento.



Serie de Sucesos

1

Diseño

El Challenger utilizaba cohetes propulsores sólidos (SRB). Con juntas selladas por anillos de goma llamados **O-rings**.

Estos anillos eran críticos: debían evitar fugas de gases.

2

Clima

28 de enero de 1986

Temperaturas inusualmente bajas en Cabo Cañaveral.

El frío volvió rígidos los **O-rings**, reduciendo su capacidad de sellado.

3

Advertencias y Deadlines

Se advirtió que el lanzamiento no debía realizarse en esas condiciones.

La presión por cumplir cronogramas llevó a la NASA a aprobar el despegue.

4

Falla durante el lanzamiento

A los pocos segundos del despegue, un O-ring falló en uno de los SRB.

Esto causó la fuga de gases calientes que comenzaron a dañar la estructura externa del transbordador.

5

Desintegración en vuelo

El chorro de gases terminó perforando el tanque externo de combustible.

A los 73 segundos, el vehículo se desintegró en el aire.

Ocasionando la muerte de los 7 astronautas.





Ejemplo del challenger

In 1986, the space shuttle Challenger exploded during takeoff, killing the seven astronauts aboard.

The explosion was the result of an O-ring failure, a splitting of a ring of rubber that seals the parts of the ship together.

The accident was believed to have been caused by the unusually cold weather (31o F or 0o C) at the time of launch, as there is reason to believe that the O-ring failure probabilities increase as temperature decreases.

Data on previous space shuttle launches and O-ring failures is given in the dataset challenger provided with the mcsn package.

The first column corresponds to the failure indicators y_i and the second column to the corresponding temperature x_i ($1 \leq i \leq 24$).

Ejercicio EN R

1. Fit this dataset with a logistic regression, where $P(Y_i = 1|x_i) = p(x_i) = \frac{\exp(\alpha + \beta x_i)}{1 + \exp(\alpha + \beta x_i)}$, using R glm function, as illustrated on page 21. Deduce the MLEs for α and β , along with standard errors.
2. Set up a Metropolis–Hastings algorithm with the likelihood as target using an exponential candidate for α and a Laplace (double-exponential) candidate for β . (Hint: Choose the parameters of the candidates based on the MLEs derived in a.)
3. Generate 5000 iterations of the Markov chain and construct a picture similar to Figure 6.6 to evaluate the variability of $p(x)$ minus the observation dots.
4. Derive from this sample an estimate of the probability of failure at 60o, 50o, and 40o F along with a standard error.

Ejercicio del challenger en R

Librería CODA,
dependencia de mcsn.

```
install.packages("coda", lib = "~/R/library", repos =  
"https://cran.r-project.org")
```

Librería MCSM, que
viene con el libro.

```
install.packages(  
"https://cran.r-project.org/src/contrib/Archive/mcsn/mcsn_1.0.tar.gz",  
repos = NULL,  
type = "source",  
lib = "~/R/library"  
)
```

Sigue en R-studio

Meet the team